

FIT2102 Tute 8: src/ Study Guide

Reader level: this guide assumes you have used Haskell very little. Every answer explains the idea from the start, shows the code, and traces an example step by step. Read the cheat sheet first.

Haskell cheat sheet (read this first)

Reading a type

```
parse :: Parser a -> String -> Maybe (String, a)
--      ^ input 1   ^ input 2   ^ the output
```

`::` means "has the type". `->` separates the inputs from the output. The last type is always the output. A lowercase letter such as `a` is a **type variable**: it means "any type". `Parser Int` is a parser whose parsed value is an `Int`.

Maybe: a value that can be missing

```
data Maybe a = Just a | Nothing
```

`Just 5` holds the value 5. `Nothing` holds no value. Haskell uses `Maybe` for results that can fail. There is no null and there are no exceptions in this code.

Function application and operators

- `f x y` calls function `f` with arguments `x` and `y`. There are no brackets and no commas.
- `f <$> p` (same as `fmap f p`): apply `f` to the value *inside* `p`.
- `pf <*> p`: `pf` holds a function. Apply it to the value inside `p`.
- `p *> q`: run `p`, then `q`. Keep the result of `q`. `p <*> q`: run both. Keep the result of `p`.
- `p <|> q`: try `p`. If it fails, try `q`.

Pattern matching

A function can have several lines. Haskell tries them from top to bottom and uses the first line whose pattern fits. `f "" = ...` matches the empty string. `f (x : xs) = ...` matches a non-empty string: `x` is the first character and `xs` is the rest.

Type class and instance

A **type class** (`Functor`, `Applicative`, `Alternative`, `Eq`, `Ord`) is a list of functions that a type can support. An **instance** says how one type supports it. Example: `instance Functor Maybe` says how `fmap` works for `Maybe`.

Result format used by every parser here

A parser returns `Just (leftover input, parsed value)` on success. The **leftover input comes first**. It returns `Nothing` on failure.

Note: the files in `supplementary/` are unfinished exercises with undefined or error stubs. Only `Functor.hs` `nestedMap` is complete.

Part 1: What each file does

Main files

1. *Parser.hs* (the core file)

```
newtype Parser a = Parser (String -> Maybe (String, a))
```

A `Parser a` is a box that holds a function. The function takes the input text. It returns `Nothing` if the parse fails. It returns `Just (remaining input, parsed value)` if it works. `parse` takes the function out of the box and runs it.

Basic parsers

- `char`: takes one character. Fails on empty input.
- `int`: reads digits at the start of the input. It uses the built-in `reads`, and it stops at the first non-digit.
- `is c`: takes one character, only if it equals `c`.
- `isNot c`: takes one character, only if it is not `c`.

The three instances (these let you combine small parsers into big parsers)

- `Functor`: `fmap f parser` changes the parsed value. The remaining input stays the same.
- `Applicative`: `pure a` always succeeds and uses no input. `p1 <*> p2` runs `p1`, then runs `p2` on what is left. If either fails, the result is `Nothing`.
- `Alternative`: `empty` always fails. `p1 <|> p2` tries `p1`. If it fails, it tries `p2` on the **same** input.

Parsers built from the basic parsers

- `string`: parses an exact word. It is recursive: `(:) <$> is x <*> string xs`.
- `whitespace` = `many (is ' ')`: zero or more spaces.
- `parseURL` = `many (isNot ' ')`: all characters up to the next space.
- `getOrPost` = `string "GET" <|> string "POST"`.
- `parseIntTuple`: parses `(3,5)` and returns `(3,5)`. `parseHTTPRequest` joins all of these. The HTTP/1.1 part is optional.

2. *Maybe.hs*

This file builds its own `Maybe` type: `data Maybe a = Just a | Nothing`. It hides the Prelude version with `import Prelude hiding (Just, Maybe, Nothing)`. It then writes the instances `Eq`, `Ord`, `Functor`, `Applicative` and `Alternative` by hand.

- `Ord`: `Nothing` is less than any `Just`. Two `Just` values compare by their contents.
- `<*>`: gives a `Just` only if both sides are `Just`.
- `<|>`: gives the first `Just`. If the first side is `Nothing`, it gives the second side.

3. *ApplicativeFunctions.hs*

This file rebuilds standard helpers from only `pure`, `<*>` and `<|>`:

- `liftA f a = pure f <*> a`
- `liftA2 f a b = pure f <*> a <*> b`
- `(<*)` = `liftA2 const`: run both, keep the left value.

- `optional a = fmap Just a <|> pure Nothing`: it never fails.

Supplementary files (unfinished exercises)

- **Base.hs**: a very small Prelude. It defines the types `RoseTree`, `Id` and `Pair`.
- **Functor.hs**: your own `Functor` class with `<$>` as the method. Laws: identity and composition. Only `nestedMap f = ((f <$>) <$>)` is finished. The instances for `Id`, `Pair`, `[]`, `Maybe`, `(->)` `r`, `(,)` `a`, `RoseTree` are stubs.
- **Applicative.hs**: your own `Applicative` class with `pure` and `<*>`. Its superclass is `Functor`. All instances are stubs.
- **Examples.hs**: stubs `sortList` (hint: `sortOn`), `groupByEqual` (hint: `groupBy` and `on`) and `nestedMap`.
- **BinTree.hs**: parses a tree from text. `L` is a leaf. `(<value><left><right>)` is a node. The parsers are stubs. `pretty` prints the tree with `|` - indents.

Part 2: How the program runs (trace this one)

Input: `parse parseHTTPRequest "POST/homepage HTTP/1.1"`

```
parseHTTPRequest =  
  (,) <$> (getOrPost <* whitespace>  
    <*> (parseURL <* optional (whitespace *> string "HTTP/1.1"))
```

Read it in two halves. The left half gives the method. The right half gives the URL. `(,)` builds a pair from the two results.

1. `getOrPost`. It tries `string "GET"`. `is 'G'` sees 'P' and fails. `<|>` then tries `string "POST"` on the same input. It matches. Leftover: `"/homepage HTTP/1.1"`. Result so far: `"POST"`.

2. `whitespace`. `many (is ' ')` finds no space at the start. `many` accepts zero matches, so it succeeds with `""`. Nothing is consumed. The `<*` keeps the result of `getOrPost`.

3. `parseURL`. `many (isNot ' ')` takes characters until it sees a space. Result: `"/homepage"`. Leftover: `"HTTP/1.1"`.

4. `optional (...)`. `whitespace` takes the space. `string "HTTP/1.1"` matches. Leftover: `""`. The `<*` drops this value and keeps the URL.

5. `(,)`. The two results combine: Just `( "", ("POST", "/homepage") )`.

If the protocol is missing, `string "HTTP/1.1"` fails, so `optional` returns `Nothing` and uses no input. The whole parse still succeeds.

Part 3: Sample questions and model answers

Difficulty tags: [easy] recall one fact, [medium] explain or trace, [hard] combine several ideas.

Section A: Theory

Q1. [easy] What is a Functor? State the two laws.

Short answer: A Functor is a container-like type that supports `fmap`. `fmap` applies a function to the values inside, and it keeps the shape the same.

Step 1: the idea. Think of `Maybe Int`, `[Int]` or `Parser Int` as boxes that hold an `Int`. You want to change the `Int` without opening the box by hand. `fmap` does that.

```
class Functor f where
  fmap :: (a -> b) -> f a -> f b
```

Read the type: give `fmap` a function from `a` to `b` and a box of `a`. You get a box of `b`. `f` is the box type (`Maybe`, `[]`, `Parser`).

Step 2: examples.

```
fmap (+1) [1,2,3]    == [2,3,4]      -- list: apply to every element
fmap (+1) (Just 5)    == Just 6       -- Maybe: apply inside Just
fmap (+1) Nothing     == Nothing      -- Maybe: nothing inside, so nothing changes
```

Step 3: the two laws. A Functor must follow them, so `fmap` never does anything surprising.

- **Identity:** `fmap id == id`. Mapping the do-nothing function `id` changes nothing.
- **Composition:** `fmap (f . g) == fmap f . fmap g`. Mapping two functions in one pass gives the same result as mapping them one after the other.

The compiler does not check these laws. The programmer must make sure they hold.

Q2. [medium] What does Applicative add to Functor? Give the types.

Short answer: `pure` puts a plain value into the box. `<*>` applies a function that is *inside* a box to a value that is inside a box. This lets you use functions with many arguments.

```
class Functor f => Applicative f where
  pure  :: a -> f a
  (<*>) :: f (a -> b) -> f a -> f b
```

Why Functor is not enough. `fmap` takes a plain function and one box. Try `fmap (+) (Just 1)`. The `(+)` takes two arguments, so it uses one argument and gives `Just (1 +)`. That is a function inside a `Just`. `fmap` cannot apply it to a second box.

How Applicative fixes this. `<*>` takes a boxed function and a boxed value:

```
Just (1 +) <*> Just 2      == Just 3
pure (+) <*> Just 1 <*> Just 2 == Just 3
-- step by step:
--   pure (+)              = Just (+)
--   Just (+) <*> Just 1    = Just (1 +)
--   Just (1 +) <*> Just 2  = Just 3
```

The pattern `pure f <*> a <*> b <*> c` works for any number of arguments. Functor `f =>` means every Applicative must also be a Functor.

Q3. [medium] What does Alternative add? What is the effect of `<|>` for Maybe, lists and Parser?

Short answer: Alternative adds **failure** (empty) and **choice** (`<|>`).

```
class Applicative f => Alternative f where
  empty :: f a
  (<|>) :: f a -> f a -> f a
```

- **Maybe:** `empty = Nothing`. `<|>` gives the first `Just`. `Nothing <|> Just 2 == Just 2`. `Just 1 <|> Just 2 == Just 1`.
- **List:** `empty = []`. `<|>` is `++`. `[1, 2] <|> [3] == [1, 2, 3]`. All choices are kept.
- **Parser:** `empty` is a parser that always fails. `p1 <|> p2` runs `p1`. If it returns `Nothing`, it runs `p2` on the **original** input. Failed attempts do not use up input.

Use `<|>` when you want to say "parse this, or else parse that".

Q4. [medium] Why is Parser a newtype of a function and not a plain type synonym?

Short answer: A newtype makes a brand-new type, so you can write your own `Functor`, `Applicative` and `Alternative` instances for it.

Step 1: what a synonym is. type `Parser a = String -> Maybe (String, a)` only gives another name to a function type. The compiler still sees `String -> ...`.

Step 2: the problem. Haskell already has instance `Functor ((->) r)` for all functions. With a synonym, your new instance would clash with it. A synonym also cannot be partly applied, so it cannot be a `Functor` argument.

Step 3: the fix. newtype `Parser a = Parser (String -> Maybe (String, a))` is a separate type with its own constructor `Parser`. No old instance applies to it, so you write your own.

A newtype costs nothing at run time. The wrapper disappears when the program is compiled. That is why `parse (Parser p) input = p input` is used to unwrap it.

Q5. [medium] What is the effect of the Parser applicative?

Short answer: There are two parts: failure spreads, and the leftover input flows from one parser to the next.

- **Failure spreads:** if one parser returns `Nothing`, the whole combined parser returns `Nothing`.
- **State flows:** the leftover input of the left parser is the input of the right parser. Text is used up from left to right.

```
parse (is '(' *> int) "(42)"
-- is '(' takes '('      leftover "42)"
-- int reads from "42)"  gives 42, leftover ""
-- result: Just (")", 42)

parse (is '(' *> int) "[42]"
-- is '(' sees '[' -> Nothing  (int is never run)
```

Q6. [hard] What can go wrong with many (is ' ') versus many (pure 1)?

Short answer: `many (is ' ')` stops when the parser fails. `many (pure 1)` never fails, so it runs forever.

How many works. `many p` runs `p` again and again until `p` fails. It collects the results in a list. Zero matches is allowed, so `many` itself never fails.

```
parse (many (is ' ')) " x"
-- is ' ' succeeds -> ' '      leftover " x"
-- is ' ' succeeds -> ' '      leftover "x"
-- is ' ' fails on 'x' -> stop
-- result: Just ("x", " ")
```

The infinite case. `pure 1` always succeeds and uses no input. After each success `many` asks for one more, and `pure 1` succeeds again with the same input. There is no stop. The program loops forever and uses

more and more memory.

Rule: only give many a parser that fails sometimes, or that always uses input.

Q7. [easy] Why does Maybe.hs have `import Prelude hiding (Just, Maybe, Nothing)`?

Short answer: The file defines its own Maybe. Without hiding, the Prelude names and the file names clash.

The Prelude is imported by default. It already has Maybe, Just and Nothing. If the file also defines them, the name Just could mean two things. GHC reports an "ambiguous occurrence" error. hiding removes the Prelude names from view, so only your names remain.

Q8. [easy] What does `{-# LANGUAGE InstanceSigs #-}` do?

Short answer: It lets you write a type signature inside an instance declaration.

```
instance Eq a => Eq (Maybe a) where
  (==) :: Maybe a -> Maybe a -> Bool    -- allowed with InstanceSigs
  (==) (Just x) (Just y) = x == y
  ...
```

Without the pragma, GHC rejects the signature line. The signature is not needed to run the code. It helps the reader, and GHC checks that it matches the class type.

Q9. [medium] Why is `Eq a =>` needed in instance `Eq a => Eq (Maybe a)`?

Short answer: The Just case compares the inner values with ==. That only works if the inner type has ==.

```
(==) (Just x) (Just y) = x == y      -- needs Eq a
(==) Nothing Nothing   = True        -- needs nothing
(==) _ _               = False       -- needs nothing
```

Read `Eq a => Eq (Maybe a)` as: "Maybe a can be compared, if a can be compared". So `Just 1 == Just 1` works because Int has Eq. A Maybe of a function type cannot be compared, because functions have no Eq.

Q10. [easy] (Parser.hs) What does returning Nothing signify in a parser like char?

Short answer: Nothing means the parser **failed**. The input did not match, so there is no result.

```
char :: Parser Char
char = Parser f
  where
    f ""          = Nothing          -- empty input: nothing to take
    f (x : xs) = Just (xs, x)       -- take x, give back the rest xs
```

Step 1: the type. The parsing function returns `Maybe (String, Char)`. Maybe has two cases. `Just (rest, value)` means success. Nothing means failure. The type itself says "this can fail".

Step 2: examples.

```
parse char "abc" == Just ("bc", 'a')  -- took 'a', left "bc"
parse char ""    == Nothing           -- no character to take
```

char fails only on empty input. Other parsers fail for other reasons. `is 'c'` fails on "abc" because the first character is 'a'. `int` fails on "abc" because there is no digit.

Step 3: why it matters. Nothing is a normal value. It is not a crash. Other code reads it and reacts:

- In `<*>`, a Nothing from the left parser becomes Nothing for the whole result.
- In `<|>`, a Nothing means "try the next option on the same input".
- In `parseHTTPRequest`, one Nothing makes the full request Nothing.

Common mistake: Nothing does not mean "parsed an empty value". `parse (string "") "abc"` succeeds with `Just ("abc", "")`. An empty result is still a success.

Q11. [medium] (Refactor.hs) Why are the guard conditions evaluated in descending order in grade?

Short answer: Guards run **top to bottom** and Haskell uses the **first True** guard. The ranges overlap, so the order decides the answer. Descending order makes every guard correct and needs only one comparison per line.

```
grade :: Int -> String
grade n
  | n >= 80  = "HD"
  | n >= 70  = "D"
  | n >= 60  = "C"
  | n >= 50  = "P"
  | otherwise = "N"
```

What is a guard? A guard is a line that starts with `|` and holds a `Bool` condition. If it is `True`, Haskell returns the value after `=` and stops. If it is `False`, Haskell goes to the next line. `otherwise` is another name for `True`. It catches all remaining cases.

Why the ranges overlap. The score 95 is `>= 80`. It is also `>= 70`, `>= 60` and `>= 50`. Four guards are `True`. Only the first must win.

Trace with n = 65 (descending order):

```
65 >= 80 ? False -> next line
65 >= 70 ? False -> next line
65 >= 60 ? True  -> return "C" and STOP
```

When the line `n >= 60` runs, the lines above it were all `False`, so `n` is already below 70. The line means "60 to 69" without writing both bounds.

What goes wrong with ascending order:

```
grade n
  | n >= 50  = "P"      -- True for 50 and EVERYTHING above
  | n >= 60  = "C"      -- never reached when n >= 50
  | n >= 70  = "D"      -- never reached
  | n >= 80  = "HD"     -- never reached
  | otherwise = "N"

grade 95 == "P"          -- wrong! should be "HD"
```

The lower lines are **dead code**: they can never run. GHC does not warn you, because the code is valid. It is only wrong.

Other order is possible only if you write both bounds on every line (`n >= 50 && n < 60`). That is longer and easier to get wrong.

Same rule elsewhere in the file: in `avgPositives` the guard `null positives` goes first. When it is `False`, the list has items, so the division in the `otherwise` line cannot divide by zero.

Rule: put the strictest condition first and `otherwise` last. For `>=` thresholds, go from the highest value down.

Section B: Predict the output

Method for every question: write the input, run the parser one step at a time, track the **leftover** input, and stop at the first failure.

Q12. [easy] `parse char "abc"` and `parse char ""`

Short answer: Just ("bc", 'a') and Nothing.

char takes the first character and gives back the rest. On "abc" it takes 'a' and leaves "bc". On the empty string there is no character, so it fails with Nothing.

Q13. [easy] parse int "123abc" and parse int "abc"

Short answer: Just ("abc", 123) and Nothing.

int calls reads. reads reads the longest number at the start of the text. On "123abc" it reads 123 and leaves "abc". On "abc" there is no number at the start, so reads returns an empty list. The _ -> Nothing case then runs.

Q14. [medium] parse (is 'h' <|> is 'w') "world, hello!"

Short answer: Just ("orld, hello!", 'w').

1. is 'h' looks at 'w'. 'w' is not 'h'. It returns Nothing.
2. <|> sees Nothing, so it runs is 'w' on the **original** input "world, hello!".
3. 'w' equals 'w'. The result is Just ("orld, hello!", 'w'). The first parser used no input, because it failed.

Q15. [medium] parse (string "abc") "abxdef" and parse (string "") "anyinput"

Short answer: Nothing, then Just ("anyinput", "").

First. string "abc" expands to (:) <\$> is 'a' <*> ((:) <\$> is 'b' <*> ((:) <\$> is 'c' <*> pure "").

```
is 'a' on "abxdef" -> Just ("bxdef", 'a')
is 'b' on "bxdef"  -> Just ("xdef", 'b')
is 'c' on "xdef"   -> Nothing             -- 'x' is not 'c'
-- one Nothing makes the whole result Nothing
```

Second. string "" = pure "". pure always succeeds and uses no input. The result is Just ("anyinput", "").

Q16. [medium] parse parseIntTuple "(33,5)" and parse parseIntTuple "(33,5,4)"

Short answer: Just ("", (33,5)), then Nothing.

```
parseIntTuple = (,) <$> (is '(' *> int) <*> (is ',' *> int <* is ')')
```

Input "(33,5)":

```
is '('      takes '('      leftover "(33,5)"
int         reads 33       leftover ",5)"
is ','      takes ','      leftover "5)"
int         reads 5        leftover ")"
is ')'      takes ')'      leftover ""
(,) 33 5    -> Just ("", (33,5))
```

Input "(33,5,4)": the steps are the same until the second int reads 5. The leftover is ",4)". Now is ')' sees ',' and fails. The whole result is Nothing.

Q17. [medium] parse parseHTTPRequest "PUT /update HTTP/1.1" and parse parseHTTPRequest "GET /home"

Short answer: Nothing, then Just ("", ("GET", "/home")).

PUT: getOrPost tries string "GET" (fails at 'P') and string "POST" (matches 'P', then fails at 'U'). Both fail, so the result is Nothing.

GET /home:

```
getOrPost      -> "GET"      leftover " /home"
whitespace     -> takes " "   leftover "/home"
parseURL       -> "/home"    leftover ""
optional (whitespace *> string "HTTP/1.1")
               -> string fails on "", so optional gives Nothing
                  and uses no input. This is still a success.
result: Just ("", ("GET", "/home"))
```

Q18. [medium] liftA2 (+) [1,2,3] [4,5]

Short answer: [5,6,6,7,7,8].

liftA2 f a b = pure f <*> a <*> b. For lists, fs <*> xs applies every function to every value.

```
pure (+)      = [(+)]
[(+)] <*> [1,2,3] = [(1+), (2+), (3+)]
[(1+), (2+), (3+)] <*> [4,5]
              = [1+4, 1+5, 2+4, 2+5, 3+4, 3+5]
              = [5, 6, 6, 7, 7, 8]
```

The left list is the outer loop. Each left value pairs with every right value.

Q19. [medium] [4,5] <*> [1,2,3] and Just 5 <*> Nothing

Short answer: [4,4,4,5,5,5], then Nothing.

(<*>) = liftA2 const. const a b = a, so the left value is kept and the right value is dropped. The right effect still happens: for lists, the effect is "do it for every right value".

```
[4,5] <*> [1,2,3] = [const 4 1, const 4 2, const 4 3,
                    const 5 1, const 5 2, const 5 3]
                  = [4,4,4,5,5,5]
```

Just 5 <*> Nothing: the right side is Nothing, so <*> cannot get a value and the whole result is Nothing.

Q20. [hard] optional [1,2] and optional Nothing

Short answer: [Just 1, Just 2, Nothing], then Just Nothing.

optional a = fmap Just a <|> pure Nothing: try a and wrap the result in Just. Otherwise give Nothing.

```
optional [1,2]
= fmap Just [1,2] <|> pure Nothing
= [Just 1, Just 2] ++ [Nothing]      -- <|> on lists is ++
= [Just 1, Just 2, Nothing]

optional Nothing
= fmap Just Nothing <|> pure Nothing
= Nothing <|> Just Nothing          -- pure Nothing = Just Nothing
= Just Nothing                      -- first Just wins
```

optional never fails. The inner Nothing means "nothing was found". The outer Just means "the optional step worked".

Q21. [easy] Just 3 > Nothing and compare Nothing Nothing

Short answer: True, then EQ.

The Ord instance in Maybe.hs defines: compare (Just _) Nothing = GT. So Just 3 is greater than Nothing, and > gives True. compare Nothing Nothing = EQ (equal).

Q22. [easy] parse (toUpper <\$> char) "abc"

Short answer: Just ("bc", 'A').

char takes 'a' and leaves "bc". fmap changes only the parsed value: toUpper 'a' is 'A'. The leftover "bc" stays the same.

Section C: Code reading

Q23. [hard] Explain fmap f (Parser p) = Parser (fmap (fmap (fmap f)) p). Which functor is each fmap?

Short answer: There are three layers of box. Each fmap goes through one layer: the function, then Maybe, then the tuple.

Step 1: the type of p. p :: String -> Maybe (String, a). Read it from the outside in:

- Layer 1: a **function** from String (the functor (->) String).
- Layer 2: a Maybe (the functor Maybe).
- Layer 3: a **tuple** (String, a) (the functor (,) String). It maps over the second part only.

Step 2: the three fmaps, outermost first.

```
fmap g      p      -- function functor: fmap g p = g . p
  where g = fmap (fmap f)  -- runs on the Maybe result of p
fmap (fmap f) m      -- Maybe functor: goes inside Just, skips Nothing
fmap f      (r, x) -- tuple functor: (r, f x). r is unchanged
```

Step 3: trace. fmap toUpper char on "abc":

```
p "abc"                = Just ("bc", 'a')
fmap (fmap toUpper) (...) = Just (fmap toUpper ("bc", 'a'))
                        = Just ("bc", 'A')
```

For Nothing, the Maybe fmap returns Nothing, so failure stays failure.

Q24. [hard] Walk through <*> for Parser. What happens if the right parser fails?

Short answer: Run the left parser, then run the right parser on the leftover. If either fails, the result is Nothing.

```
p1@(Parser a) <*> p2@(Parser b) = Parser $ \input -> case a input of
  Nothing      -> Nothing
  Just (rest, f) -> case b rest of
    Nothing      -> Nothing
    Just (rest', x) -> Just (rest', f x)
```

\input -> ... is an anonymous function with the argument input. The left parser p1 produces a **function** f. The right parser produces a value x.

- **Step 1:** run a on input. If it gives Nothing, stop with Nothing.
- **Step 2:** otherwise you have (rest, f). Run b on rest (not on input).
- **Step 3:** if b gives Nothing, stop with Nothing.
- **Step 4:** otherwise you have (rest', x). Return Just (rest', f x).

If the right parser fails, the final result is Nothing, even if the left parser worked. The right parser starts where the left one stopped. The leftover input flows left to right.

Q25. [medium] What are *> and <*>? Why does parseIntTuple use them?

Short answer: Both run two parsers in order. *> keeps the right result. <*> keeps the left result. They let you check punctuation without keeping it.

The arrow points at the side that is kept. parseIntTuple must read (, , and), but return only the two numbers:

```
(,) <$> (is '(' *> int) <*> (is ',' *> int <*> is ')')
      ^^^^^^^^^^^^^^^^^^      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
      '(' checked, 33 kept    ',' checked, 5 kept, ')' checked
```

- is '(' *> int: run is '(', drop '(', keep the number.
- is ',' *> int <*> is ')': groups as (is ',' *> int) <*> is ')': keep the number, check), drop ')'.
The arrow points at the side that is kept.
- (,) <\$> a <*> b builds the pair from the two kept numbers.

Q26. [hard] Why does string (x:xs) = (:) <\$> is x <*> string xs work?

Short answer: is x parses the first character. string xs parses the rest. (:) joins them into one string.

```
string "" = pure ""
string (x : xs) = (:) <$> is x <*> string xs
```

Step 1: types. (:) is the "add to the front of a list" function: (:) 'a' "bc" == "abc".

Step 2: first half. (:) <\$> is x maps (:) over the parser. The parsed Char becomes a function. The result has type Parser (String -> String): it still waits for the rest of the string.

Step 3: second half. <*> string xs runs the recursive parser on the leftover. It supplies the missing String.

Step 4: trace for string "ab" on "abc".

```
string "ab" = (:) <$> is 'a' <*> string "b"
             = (:) <$> is 'a' <*> ((:) <$> is 'b' <*> string "")
             = (:) <$> is 'a' <*> ((:) <$> is 'b' <*> pure "")
is 'a' -> 'a', leftover "bc"
is 'b' -> 'b', leftover "c"
pure "" -> "", leftover "c"
'b' : "" = "b"
'a' : "b" = "ab"          result: Just ("c", "ab")
```

Base case: string "" = pure "" ends the recursion. It uses no input.

Q27. [medium] nestedMap f = ((f <\$>) <\$>). Explain it.

Short answer: It applies f to the values inside **two** boxes, such as a list of Maybes.

```
nestedMap :: (Functor f, Functor g) => (a -> b) -> f (g a) -> f (g b)
```

(f <\$>) is a function that maps f over one box (the inner box g). ((f <\$>) <\$>) maps *that function* over the outer box f.

```
nestedMap (+1) [Just 1, Nothing]
= ((+1) <$>) <$> [Just 1, Nothing]
= [ (+1) <$> Just 1, (+1) <$> Nothing ]    -- outer map over the list
= [ Just 2, Nothing ]                    -- inner map over each Maybe
```

Q28. [hard] Why can writing fmap with pure and <*> cause an infinite loop?

Short answer: If <*> also calls fmap, the two functions call each other forever.

In Applicative.hs the exercise asks for `fmap f x = pure f <*> x`. This is correct **only if** `<*>` does not use `fmap`.

```
fmap f x    = pure f <*> x          -- calls <*>
(<*>) fs x  = ... fmap ... x        -- calls fmap again
-- fmap -> <*> -> fmap -> <*> -> ... (no base case)
```

This is mutual recursion with no base case, so the program never stops. **Fix:** write `<*>` with pattern matching or list comprehensions (see Q32). Do not call `fmap` inside it.

Section D: Write code

Q29. [medium] Write `liftA3` using only `pure` and `<*>`.

Short answer: `liftA3 f a b c = pure f <*> a <*> b <*> c`.

```
liftA3 :: Applicative f => (a -> b -> c -> d) -> f a -> f b -> f c -> f d
liftA3 f a b c = pure f <*> a <*> b <*> c
```

Step 1: `pure f` puts the 3-argument function into the box. **Step 2:** each `<*>` feeds one boxed argument. `<*>` is left-associative, so it reads `((pure f <*> a) <*> b) <*> c`. After each step the function has one argument less.

```
liftA3 (\x y z -> x+y+z) (Just 1) (Just 2) (Just 3)
= Just (\x y z -> ...) <*> Just 1 <*> Just 2 <*> Just 3
= Just (\y z -> 1+y+z) <*> Just 2 <*> Just 3
= Just (\z -> 1+2+z) <*> Just 3
= Just 6
```

Q30. [medium] Write a parser `digitPair :: Parser (Int, Int)` for input like "3-4".

Short answer: `digitPair = (,) <$> int <*> (is '-' *> int)`.

```
digitPair :: Parser (Int, Int)
digitPair = (,) <$> int <*> (is '-' *> int)
```

- `int` reads the first number (3). Leftover: "-4".
- `is '-' *> ...` takes the dash and drops it. Leftover: "4".
- `int` reads the second number (4).
- `(,) <$> ... <*> ...` builds the pair (3,4).

`parse digitPair "3-4" == Just ("", (3,4))`. This has the same shape as `parseIntTuple`.

Q31. [medium] Write a parser that reads "yes" or "no" and returns a `Bool`.

Short answer: `yesNo = (True <$ string "yes") <|> (False <$ string "no")`.

```
yesNo :: Parser Bool
yesNo = (True <$ string "yes") <|> (False <$ string "no")
```

`x <$ p` runs `p`, drops its result and returns the constant `x` instead. The parsed text matters only for the match.

```
parse yesNo "yes!" == Just ("!", True)
parse yesNo "no"   == Just ("", False)
parse yesNo "maybe" == Nothing      -- both options fail
```

If `string "yes"` fails, `<|>` tries `string "no"` on the same input.

Q32. [hard] Write `Applicative` for `Id` and for lists (supplementary stubs).

Short answer: `Id`: `unwrap` both, `apply`, `wrap`. `List`: `apply` every function to every value.

```

newtype Id a = Id a                                -- from Base.hs

instance Applicative Id where
  pure = Id
  Id f <*> Id x = Id (f x)

instance Applicative [] where
  pure x    = [x]
  fs <*> xs = [f x | f <- fs, x <- xs]

```

Id: pure just wraps the value. <*> matches both sides with the pattern `Id ...`, so it gets `f` and `x`, then wraps `f x`. Example: `Id (+1) <*> Id 2 == Id 3`.

List: `pure x = [x]` is a list with one item. The comprehension `[f x | f <- fs, x <- xs]` means: for each function `f` in `fs`, and for each value `x` in `xs`, make `f x`. The first generator is the outer loop.

```
[(+1), (*2)] <*> [10, 20] == [11, 21, 20, 40]
```

The superclass `Functor` instance must also exist. This definition does not call `fmap`, so there is no loop.

Q33. [hard] Write `Applicative` for `((->) r)` (functions that take an `r`).

Short answer: `pure x = \_ -> x`, and `f <*> g = \r -> f r (g r)`.

```

instance Applicative ((->) r) where
  pure x    = \_ -> x
  f <*> g    = \r -> f r (g r)

```

Types. Here the "box" is a function waiting for an input `r`. `f :: r -> (a -> b)`, `g :: r -> a`.

- `pure x` is a function that ignores its input and always returns `x`. (`const x` does the same.)
- `f <*> g` is a function that takes `r`, gives the **same** `r` to both `f` and `g`, then applies `f`'s result to `g`'s result.

Check: `((*) <*> (+1)) 2`

```

= \r -> (*) r ((+1) r)   applied to 2
= (*) 2 ((+1) 2)
= (*) 2 3
= 6

```